

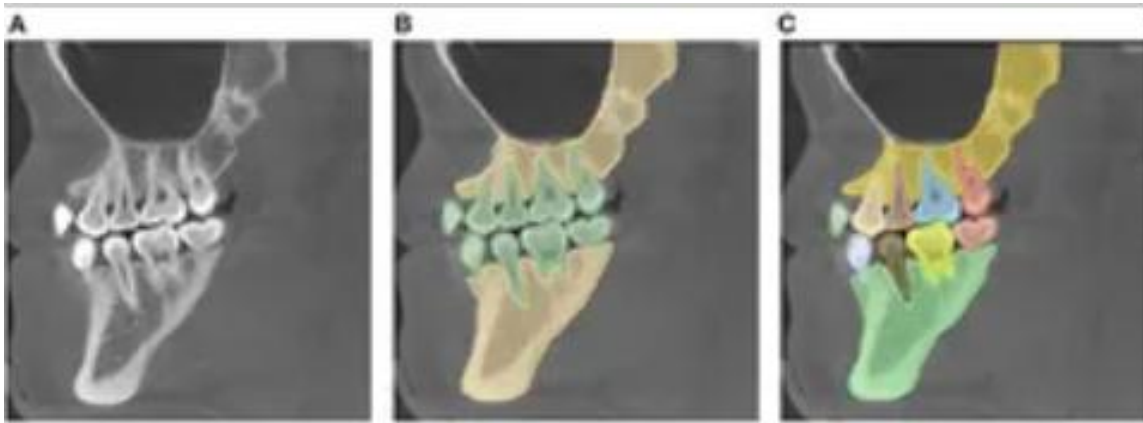
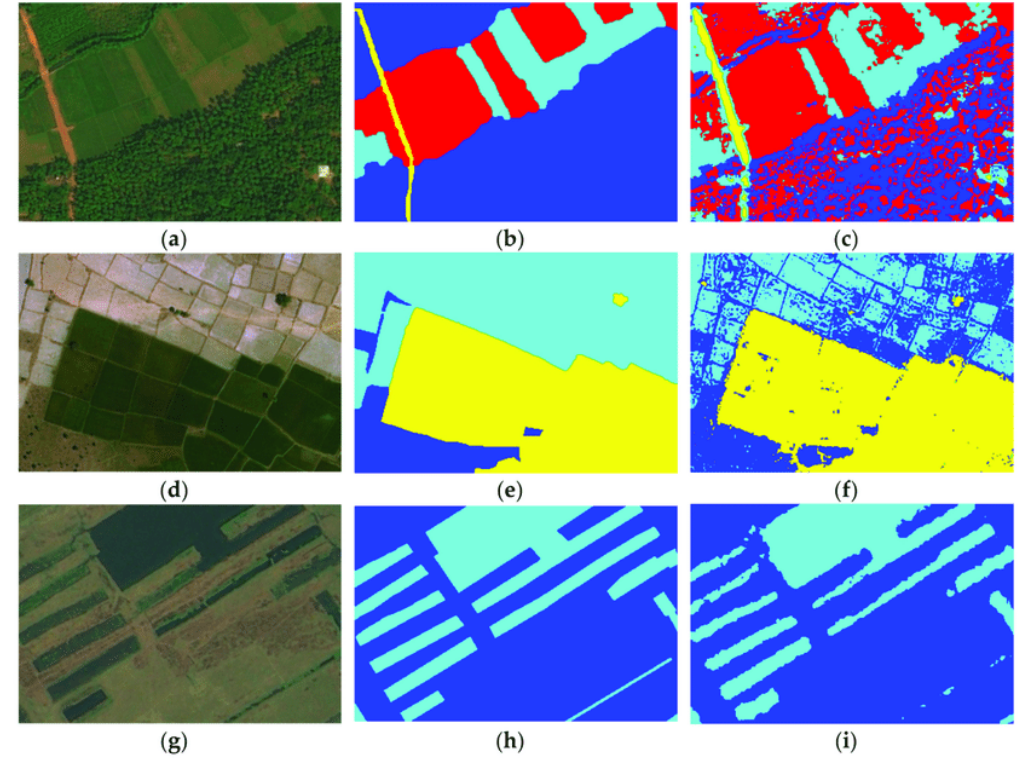
# Semantic image segmentation

# Semantic image segmentation

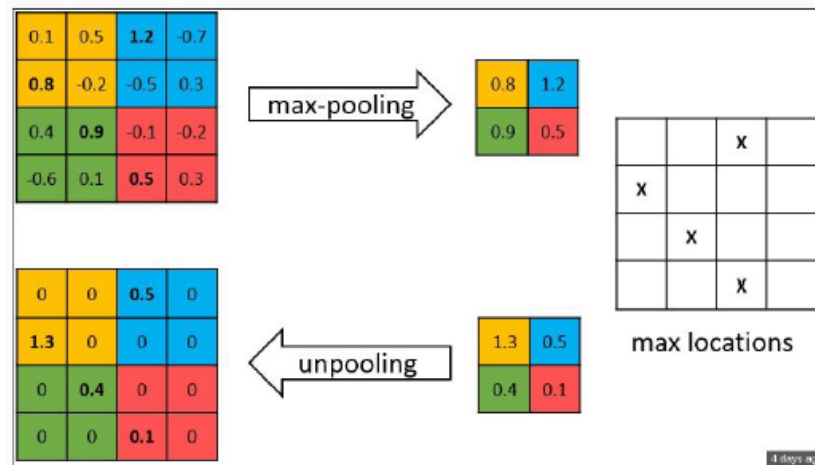
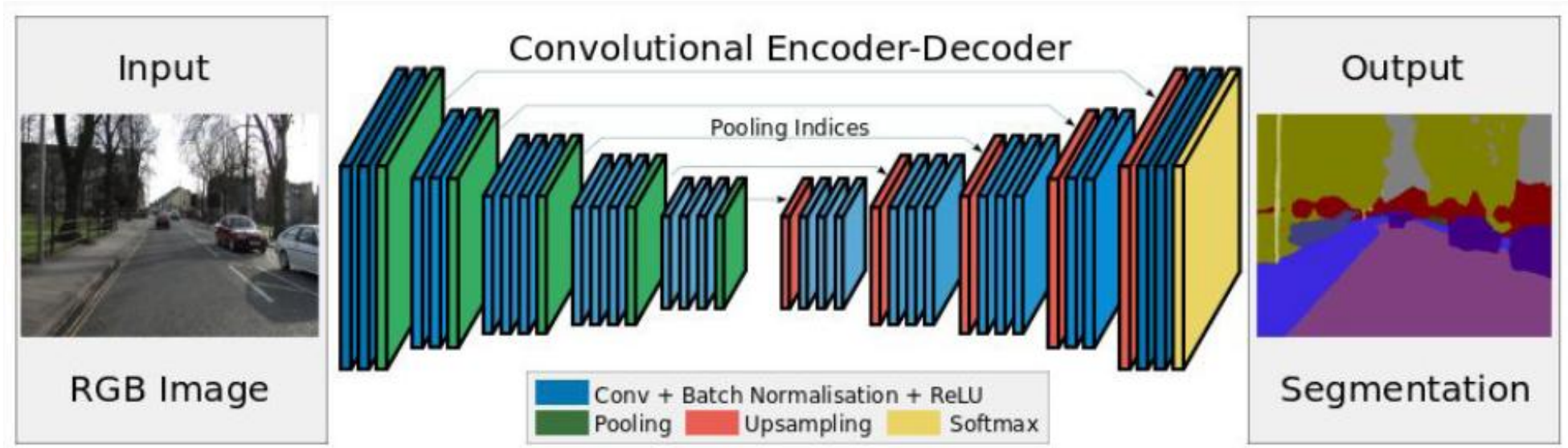


| Dataset                        | Training | Testing | #Classes |
|--------------------------------|----------|---------|----------|
| <a href="#">CamVid</a>         | 468      | 233     | 11       |
| <a href="#">PascalVOC 2012</a> | 9963     | 1447    | 20       |
| <a href="#">NYUDv2</a>         | 795      | 645     | 40       |
| <a href="#">Cityscapes</a>     | 2975     | 500     | 19       |
| <a href="#">Sun-RGBD</a>       | 10355    | 2860    | 37       |
| <a href="#">MS COCO '15</a>    | 80000    | 40000   | 80       |
| <a href="#">ADE20K</a>         | 20210    | 2000    | 150      |

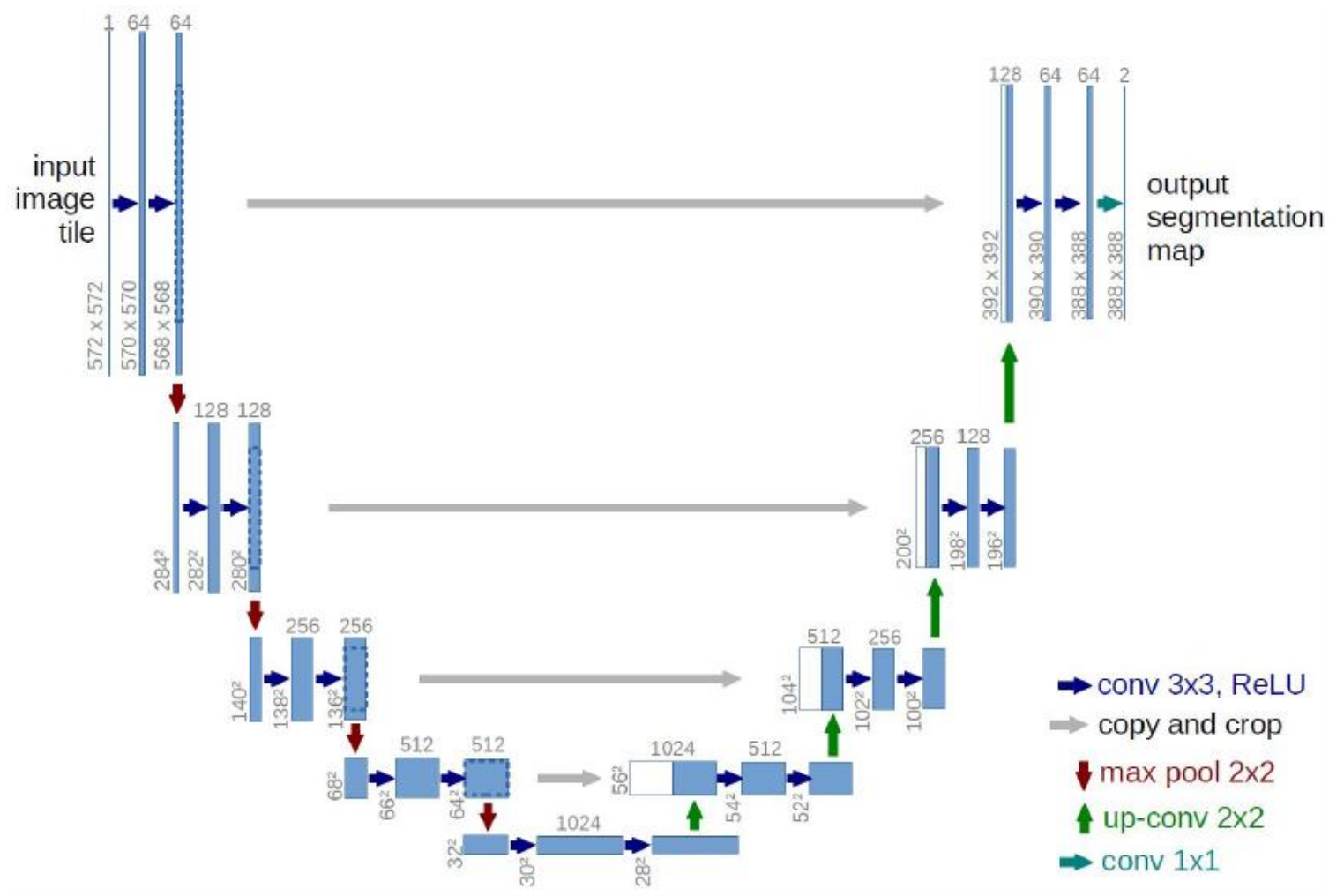
# Semantic image segmentation



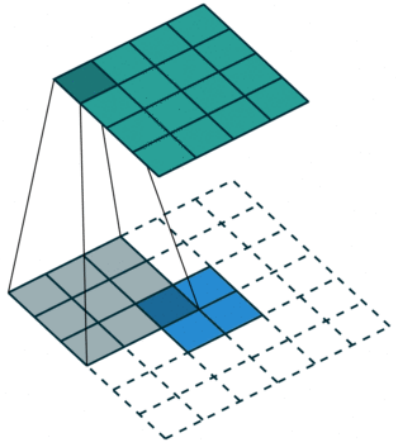
# SegNet (2015)



# U-net (2015)

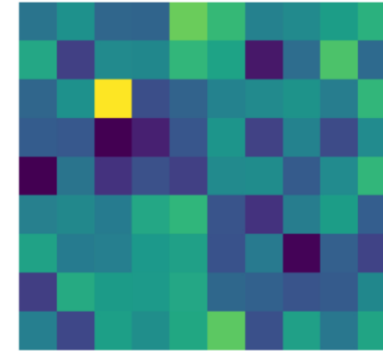
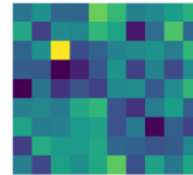
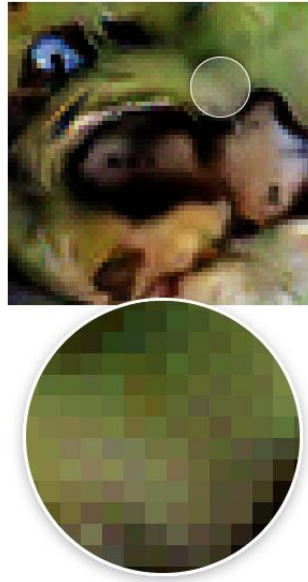
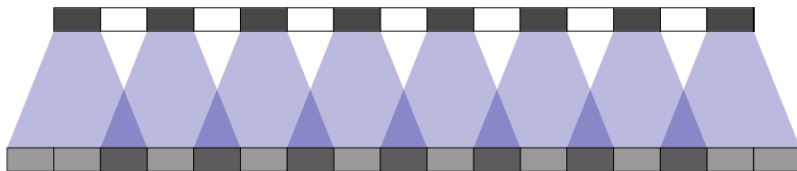


# Different up-scale transforms



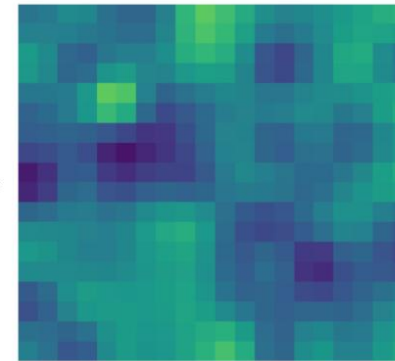
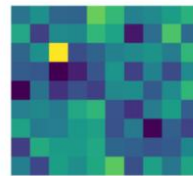
Transposed convolutions:

- has learnable parameters
- may slow to compute
- may lead sometimes to checkerboard patterns



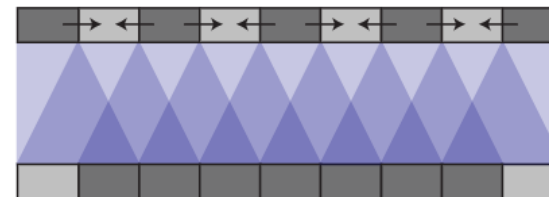
Nearest Neighbors upscaling:

- No parameters, fast to compute
- Too blocky result



Bilinear interpolation:

- No parameters, fast to compute, can both upscale and downscale the result
- Usually shows good results



An [article](#) about checkerboard effect

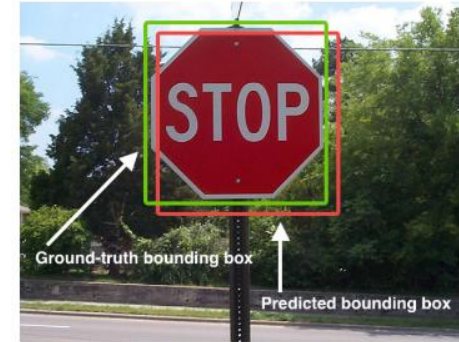
# Measuring performance for semantic segmentation

Main problem: different segments may be very imbalanced in size => standard accuracy can be misleading

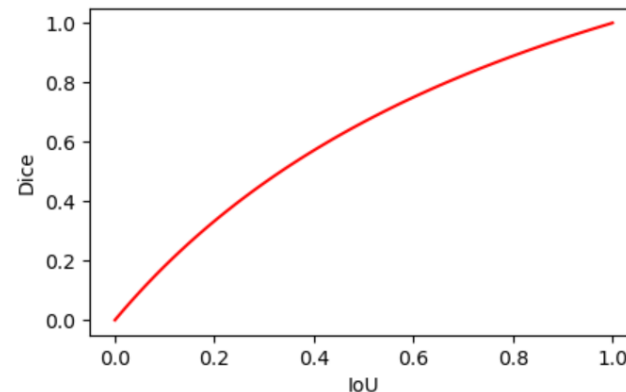
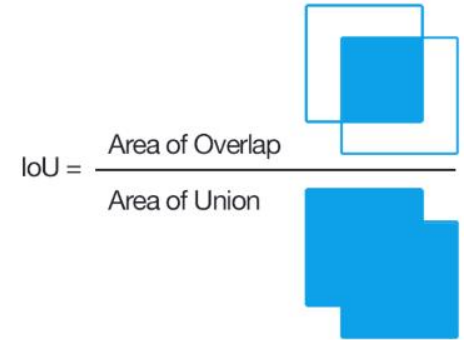
$$\text{IoU} = \frac{|Y \cap T|}{|Y \cup T|} \quad \text{Dice} = \frac{2 \cdot |Y \cap T|}{|Y| + |T|}$$

Dice and IoU are monotonically dependent:

$$\text{Dice} = \frac{2 \cdot \text{IoU}}{1 + \text{IoU}}$$



*Intersection over Union*



# Testing performance

$y_{ip}$  – true class label for image  $i$  for pixel  $p$

$p_{ipk}$  – probability prediction for image  $i$ , pixel  $p$ , class  $k$

$\hat{y}_{ip} = \arg \max_k p_{ipk}$  – predicted class label

Main metric:

$$\text{Class IoU} = \frac{1}{K} \sum_{k=1}^K \frac{\sum_{i,p} [y_{ip} = k][\hat{y}_{ip} = k]}{\sum_{i,p} [y_{ip} = k][\hat{y}_{ip} = k] + [y_{ip} = k][\hat{y}_{ip} \neq k] + [y_{ip} \neq k][\hat{y}_{ip} = k]}$$

Auxiliary metric:

$$\text{Instance IoU} = \frac{1}{N} \sum_{i=1}^N \frac{\sum_{p,k} [y_{ip} = k][\hat{y}_{ip} = k]}{\sum_{p,k} [y_{ip} = k][\hat{y}_{ip} = k] + [y_{ip} = k][\hat{y}_{ip} \neq k] + [y_{ip} \neq k][\hat{y}_{ip} = k]}$$

# Training losses

$$\text{Loss} = \text{CrossEntropyLoss} + \text{SoftDiceLoss}$$

$$\text{CrossEntropyLoss} = -\frac{1}{N} \sum_{i=1}^N \sum_p \sum_k w_k [y_{ip} = k] \log p_{ipk}$$

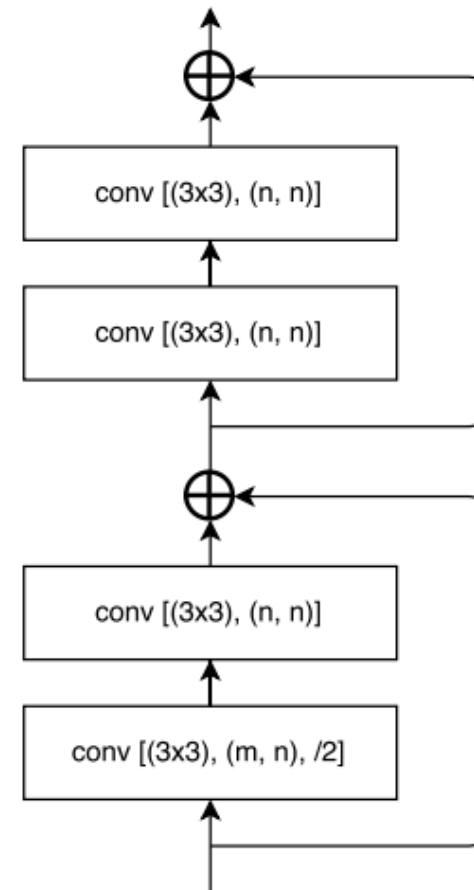
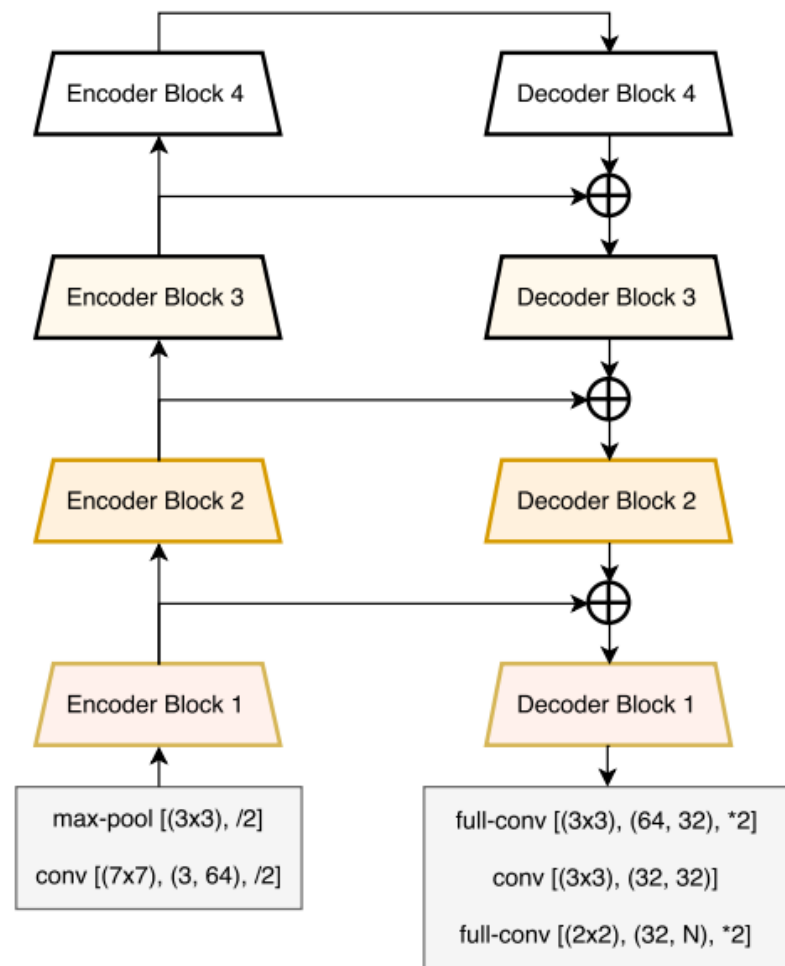
$$w_k = \frac{1}{\log(\text{prob}_k + 1.02)} \in [1, 50]$$

$$\text{SoftDice}_k(i) = \frac{2 \sum_p p_{ipk} [y_{ip} = k] + \varepsilon}{\sum_p p_{ipk} + \sum_p [y_{ip} = k] + \varepsilon}$$

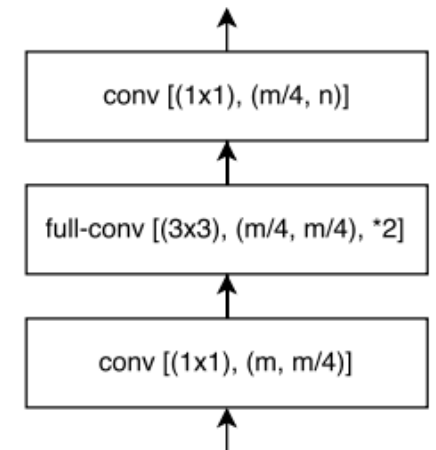
$$\text{SoftDiceLoss} = \frac{1}{N} \sum_{i=1}^N \left[ 1 - \frac{1}{K-1} \sum_{k=1}^{K-1} \text{SoftDice}_k(i) \right]$$

Background class can be excluded

# LinkNet (2017)

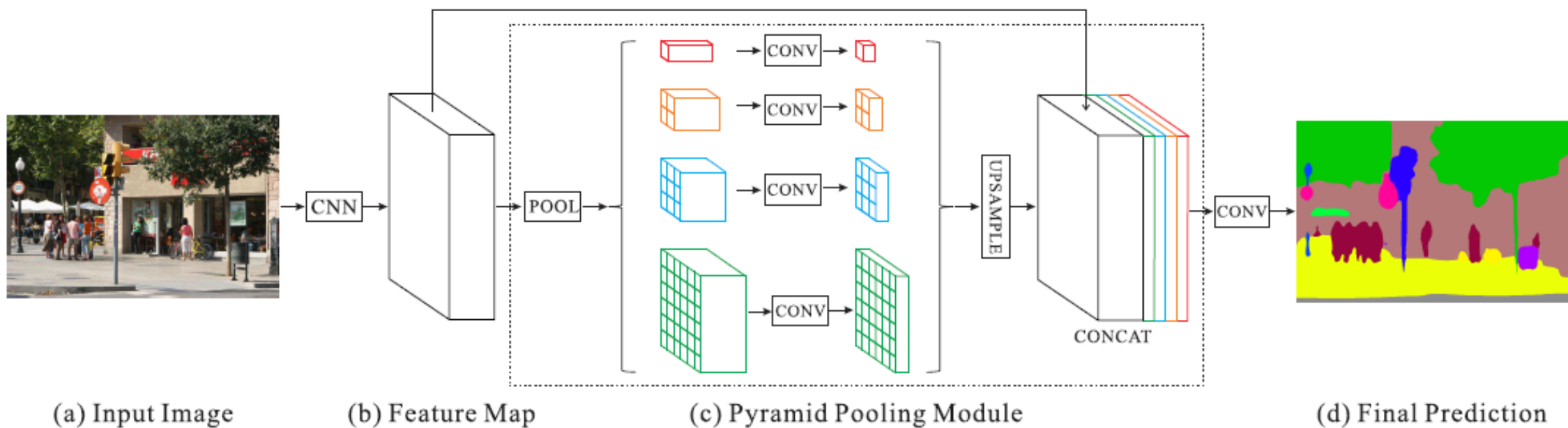


Encoder block

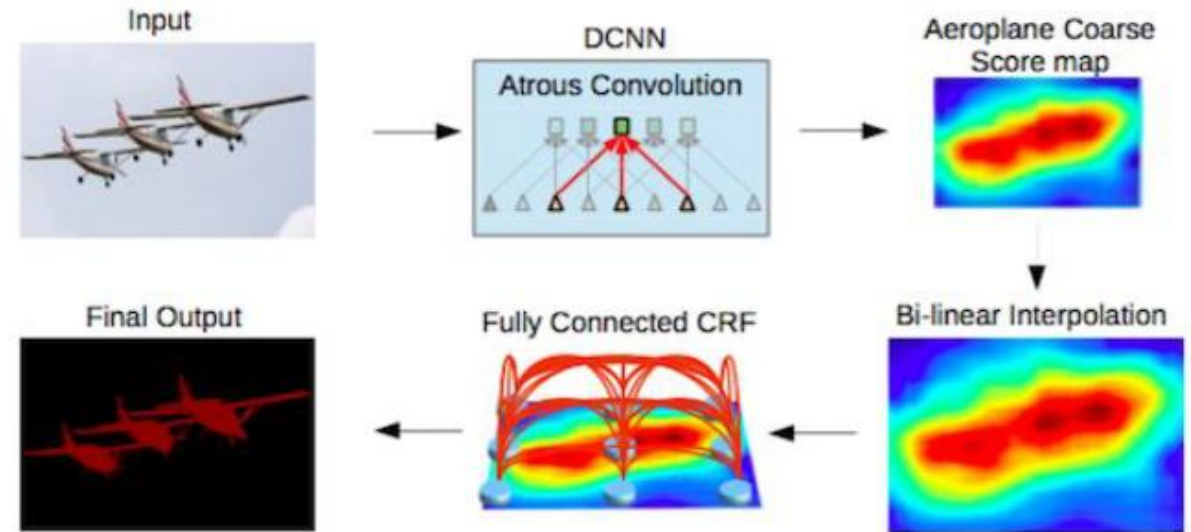
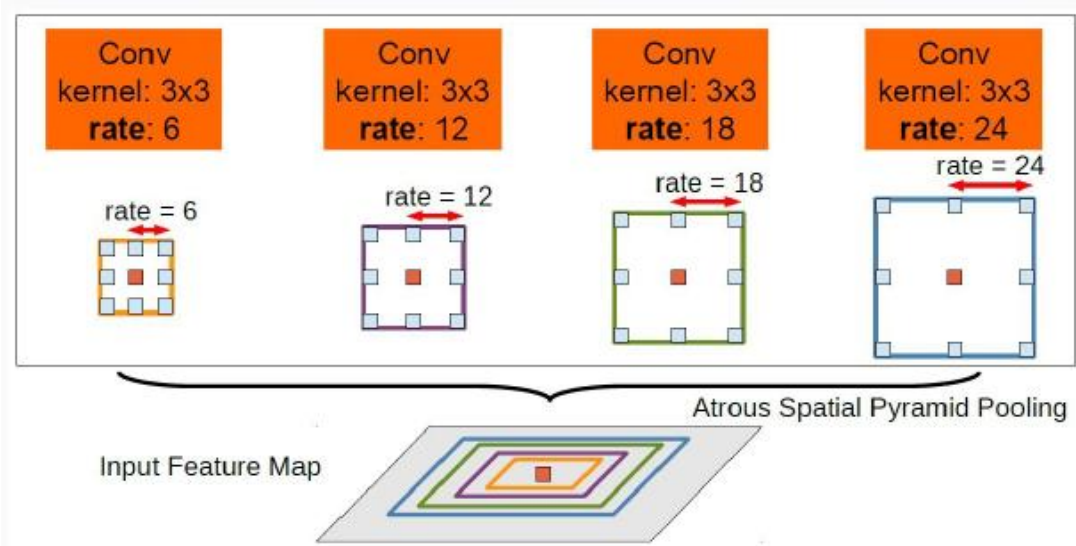


Decoder block

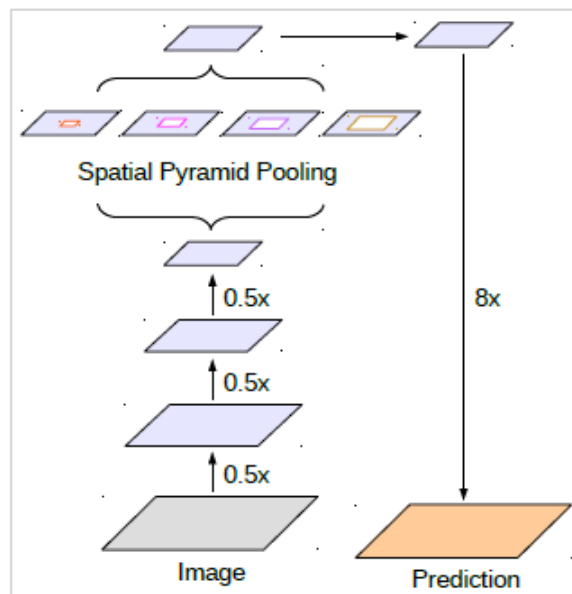
# PSPNet (2016)



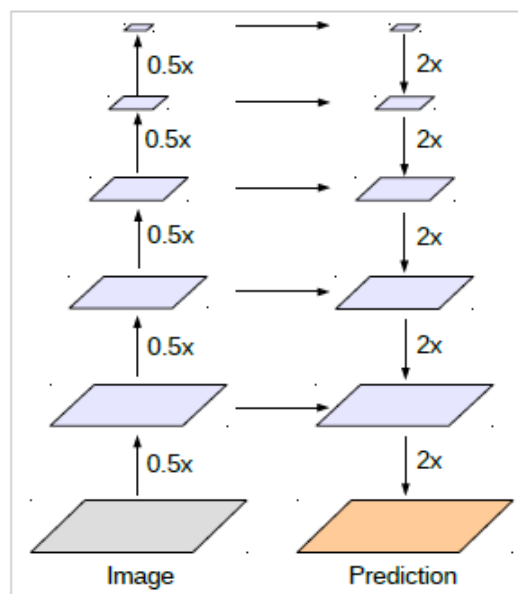
# DeepLab v1, v2, ...



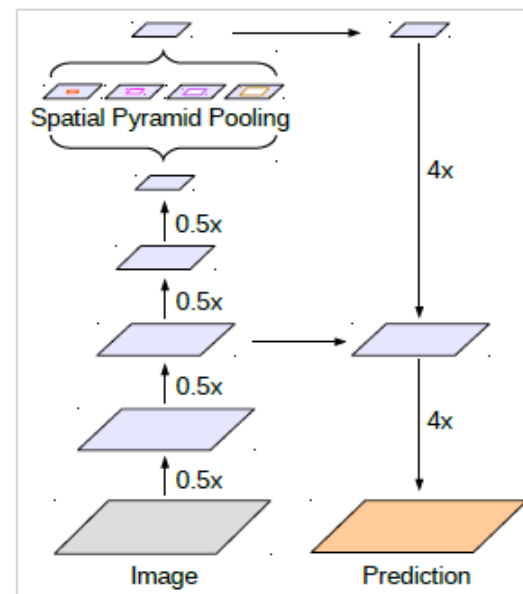
# DeepLab v3+



(a) Spatial Pyramid Pooling



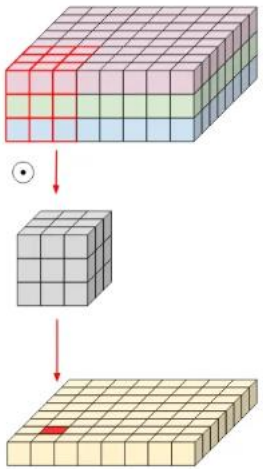
(b) Encoder-Decoder



(c) Encoder-Decoder with Atrous Conv

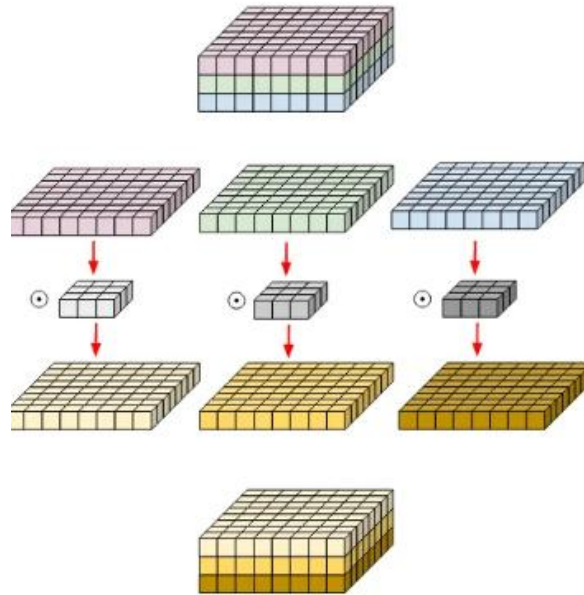
# Depth-wise separable convolution

Normal convolution



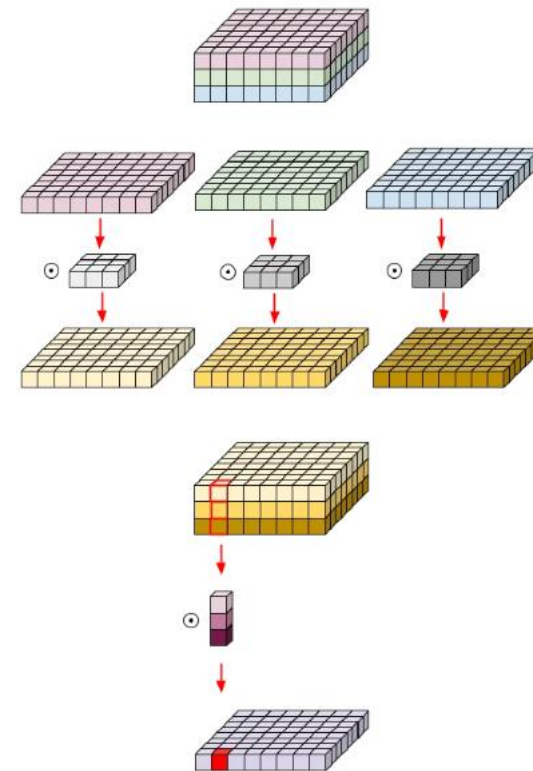
No. of params for 3 output channels:  $3 \cdot (3 \cdot 3 \cdot 3)$

Depth-wise convolution



No. of params:  $3 \cdot 3 \cdot 3$

Depth-wise separable convolution



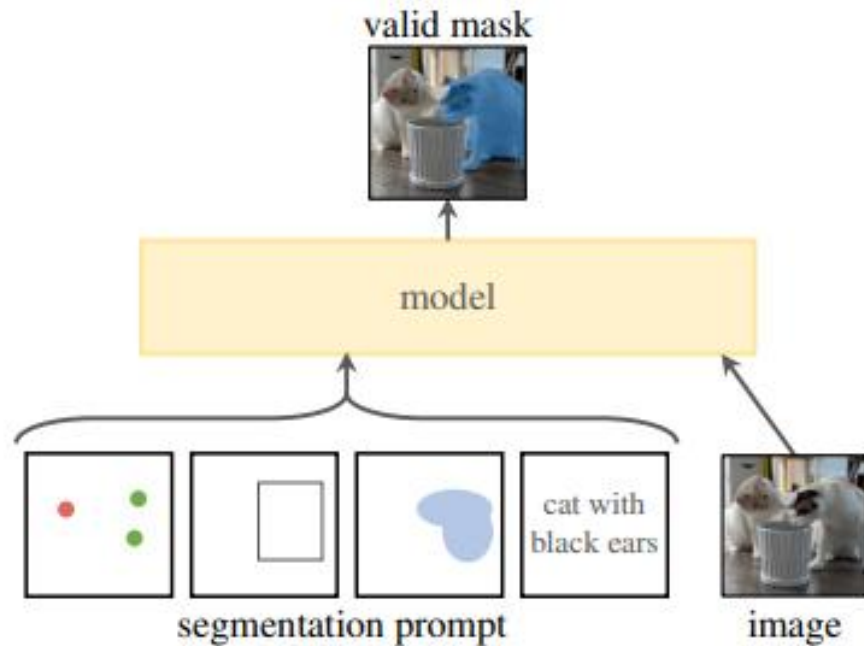
No. of params for 3 output channels:  $3 \cdot 3 \cdot 3 + 3 \cdot (3 \cdot 1)$

# Image segmentation

Main architectural ideas with convolutional NNs:

- Encoder-decoder structure with connections on different resolution levels
- Pyramid Pooling Module (PPM) or Atrous Spatial Pyramid Pooling (ASPP) for analyzing images on different scales and better consider global context
- Depth-wise separable convolutions for a smaller number of parameters and faster computations

# Segment Anything Model for interactive segmentation (SAM)



Object detection on images

# Object detection



# Measuring detection performance

Many sources of errors in detection: classification errors, localization errors, duplicates, non-found bounding boxes, etc.

$$\text{Precision} = \frac{TP}{TP + FP}$$

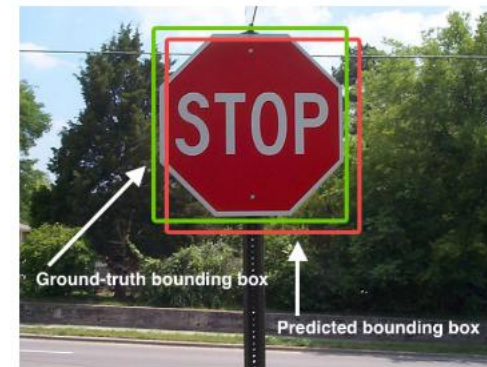
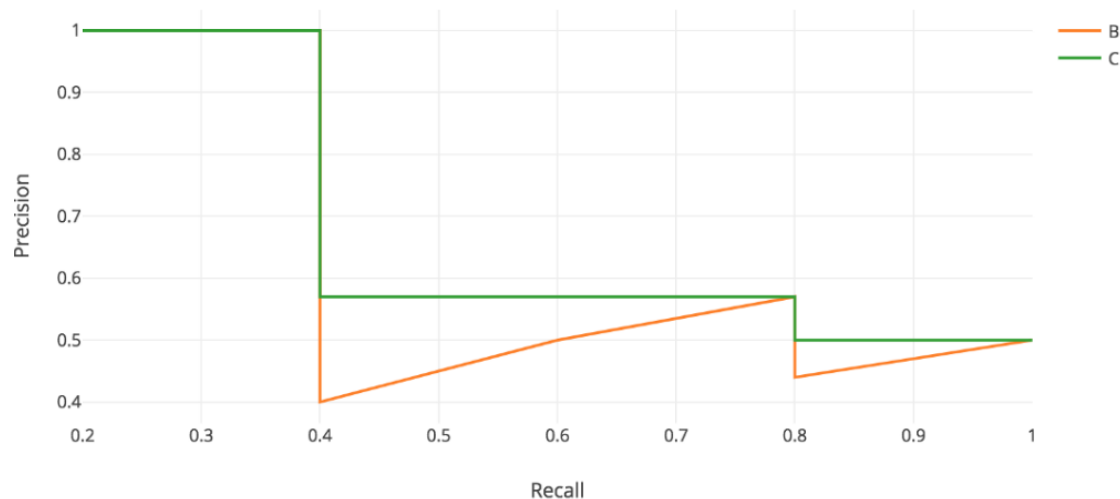
$$\text{Recall} = \frac{TP}{TP + FN}$$

$TP$  = True positive

$TN$  = True negative

$FP$  = False positive

$FN$  = False negative



*Intersection over Union*

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

A diagram illustrating the Intersection over Union (IoU) calculation. It shows two overlapping blue rectangles. The top rectangle is slightly offset to the right and up from the bottom rectangle. The area where they overlap is shaded in a darker blue. The formula above the diagram states: IoU = Area of Overlap / Area of Union.

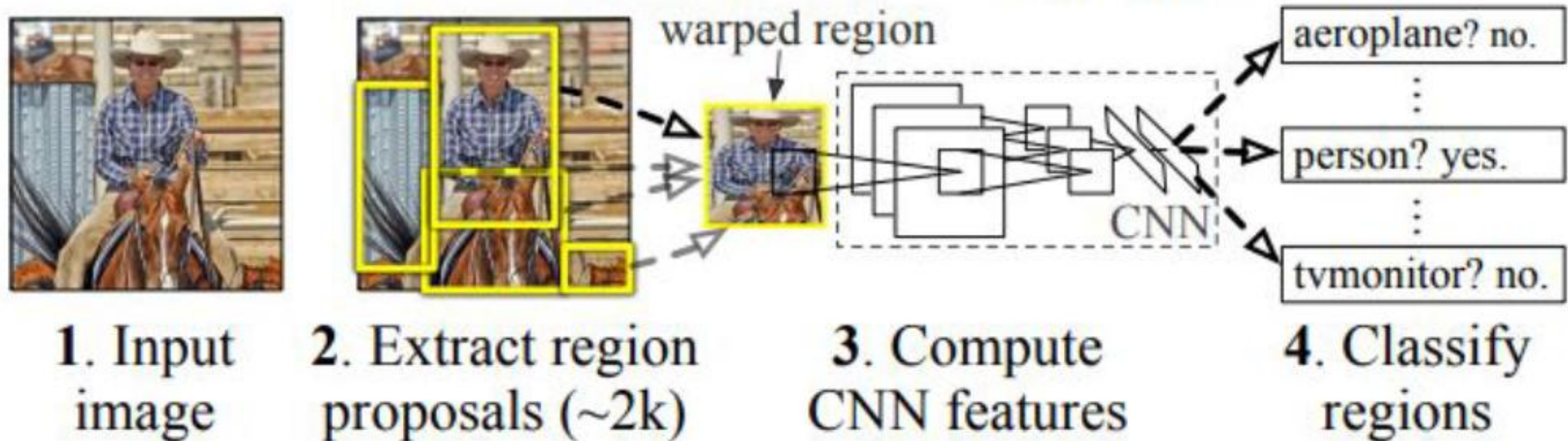
# Measuring detection performance

## mAP computation scheme

1. For each class: for all testing images take all bounding-boxes predictions for given class with their classification scores.
2. Init:  $TP = 0$ ,  $FP = 0$ ,  $FN = \#GT \text{ BBs for this class}$
3. Sort all BBs w.r.t. classification scores and sequentially compute:
  - 3.1. If BB has  $IoU \geq 0.5$  with GT BB for this class and this GT BB is not assigned to some predicted BB, then  $TP+=1$ , assign GT BB to given BB,  $FN-=1$
  - 3.2. If BB has  $IoU \geq 0.5$  with GT BB and this GT BB is already assigned (duplicate) or  $IoU < 0.5$ , then  $FP+=1$
  - 3.3. Compute Precision, Recall values
4. Compute  $AP = \text{Area under PR-curve}$
5. Compute  $mAP = AP$  averaging w.r.t. all classes
6. Compute  $mAP@[0.5:0.95]$  by averaging mAP at different thresholds for IoU.

# R-CNN (2013)

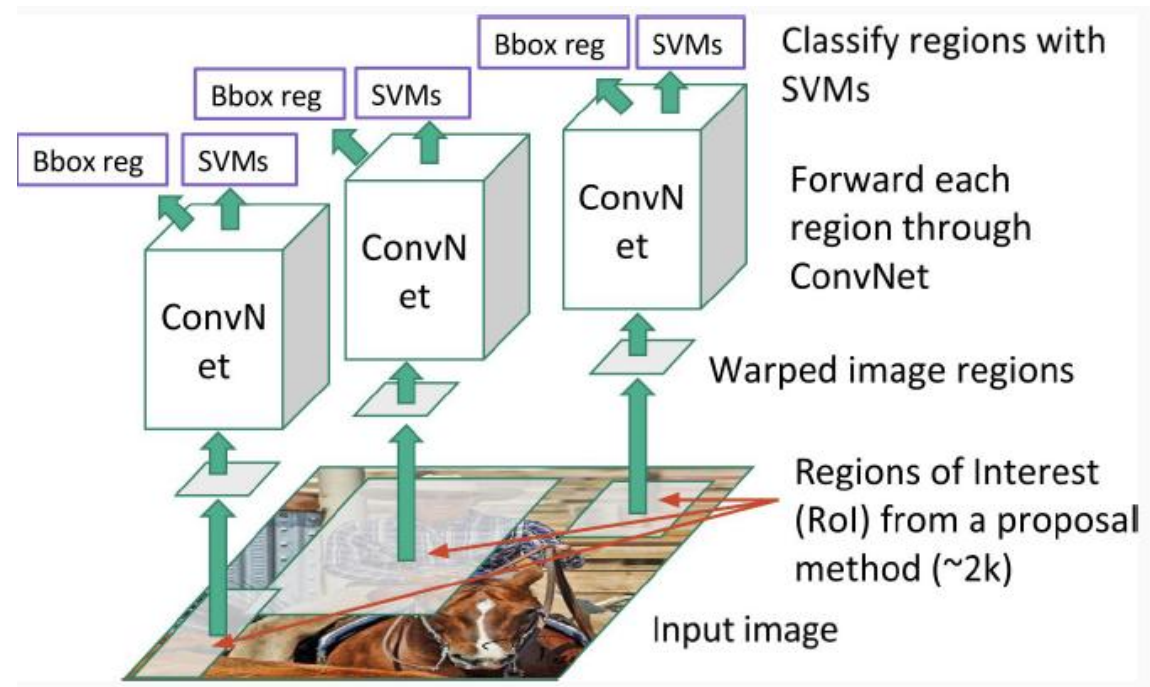
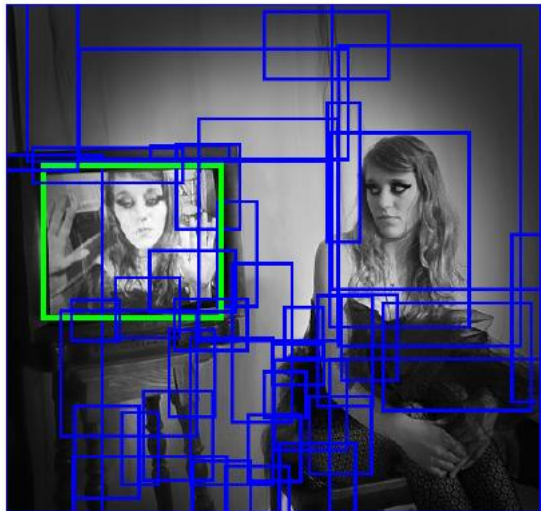
## R-CNN: *Regions with CNN features*



# R-CNN (2013)

Training:

- 1) Generation of region proposals using Selective Search (classic CV algorithm = agglomerative clustering on superpixels)
- 2) Training classifiers and regressors



# R-CNN (2013)

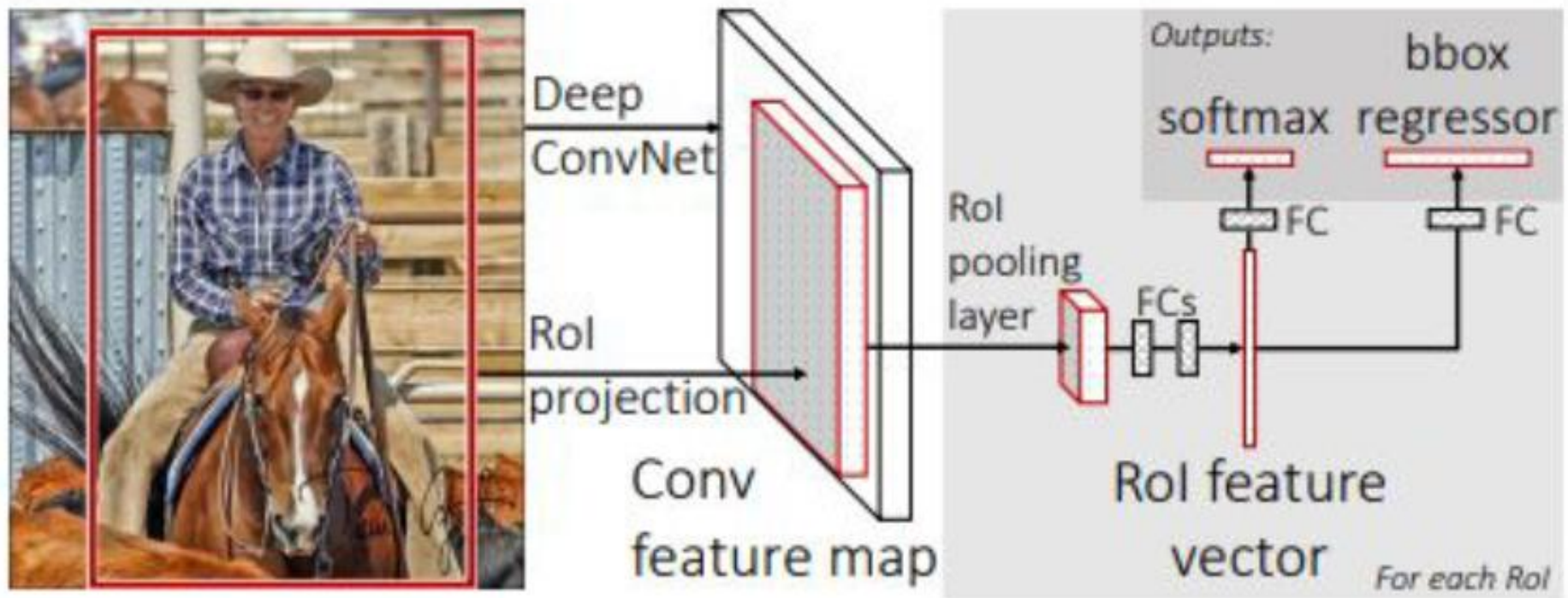
Inference:

- 1) Generate region proposals
- 2) For each region find class probabilities and regression correction coefficients
- 3) For each class make greedy non-maximum suppression

# Fast R-CNN (2015)

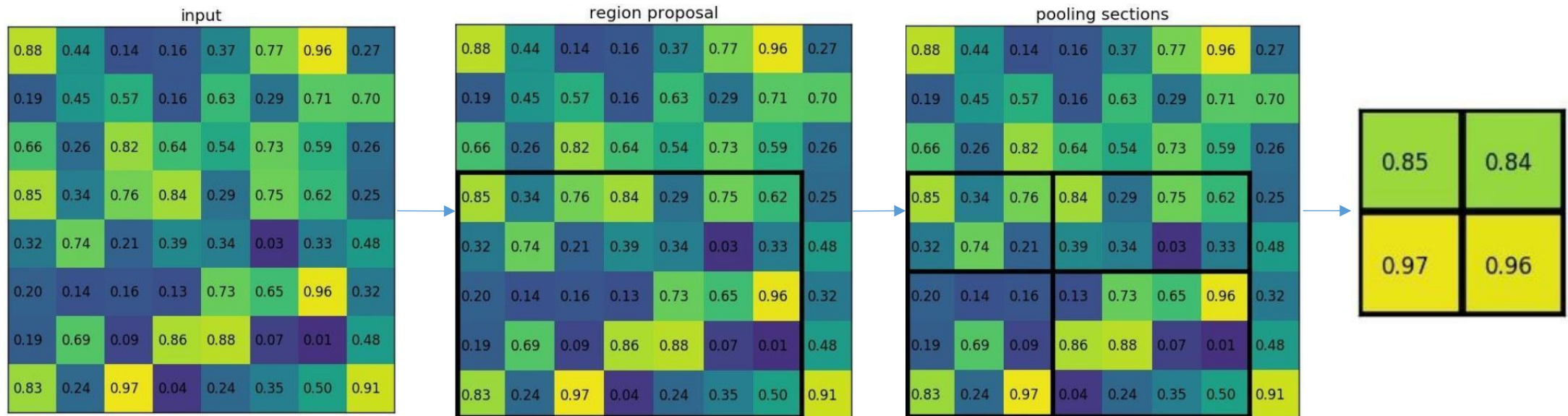
Main idea: CNN is applied for each image only once

Training in one stage

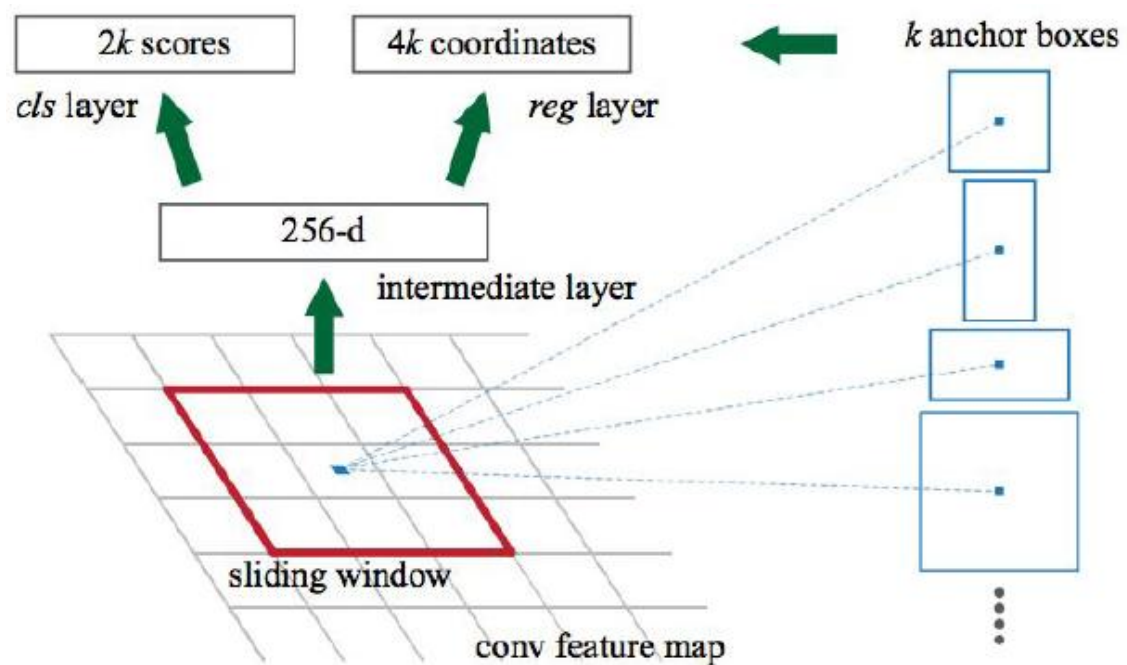
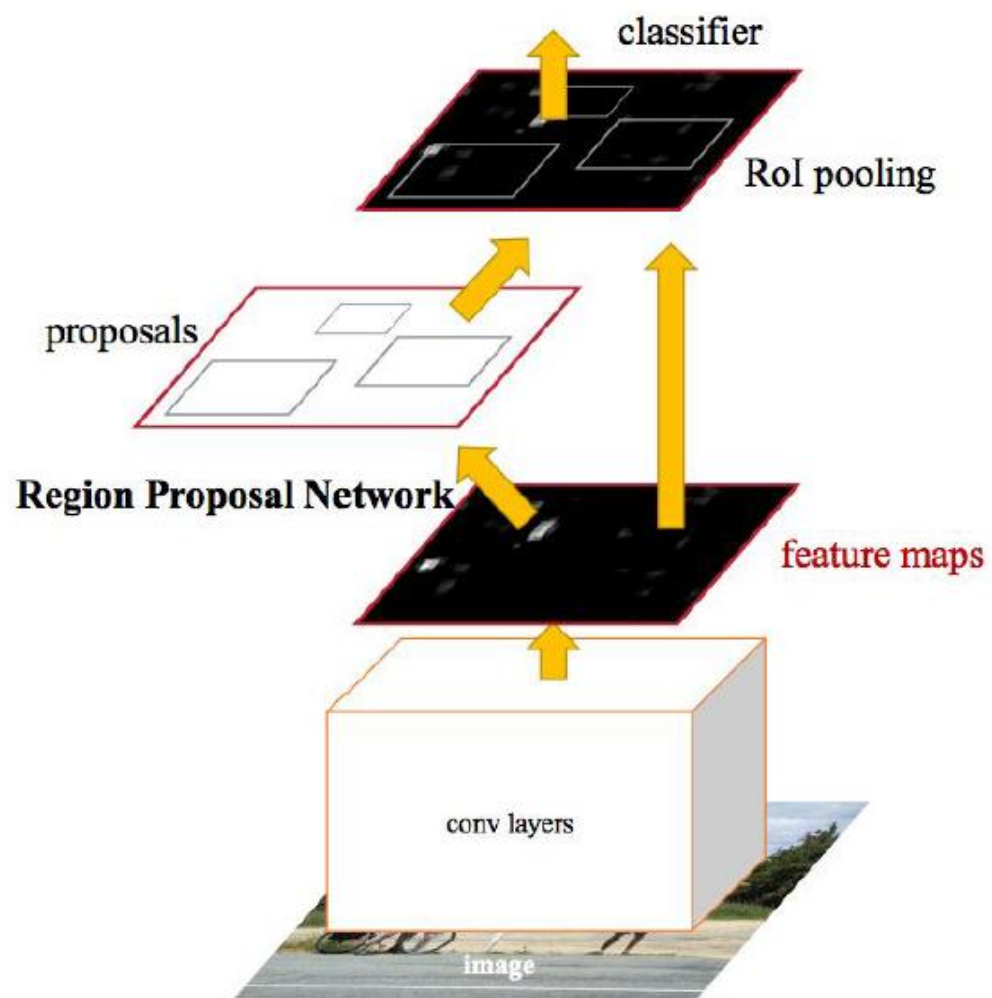


# Fast R-CNN (2015)

Finding fixed-sized vector for each RoI



# Faster R-CNN (2015)



# Faster R-CNN (2015)

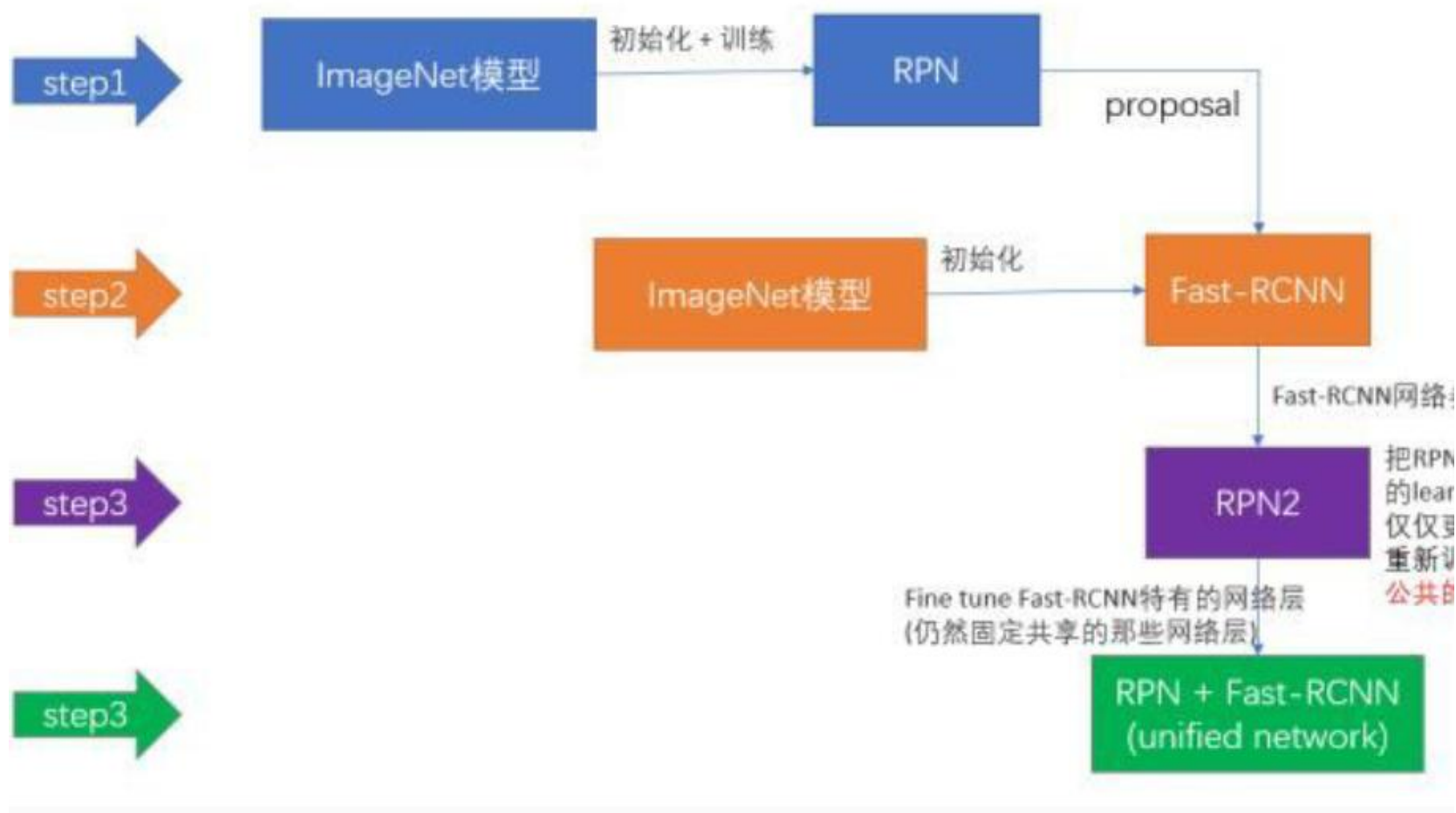
RPN training:

- 1) Find all anchors
- 2) Skip/truncate anchors intersecting image boundaries
- 3) Determine positive and negative anchors using IoU with true bounding boxes
- 4) Train object presence classifier and bbox regressor

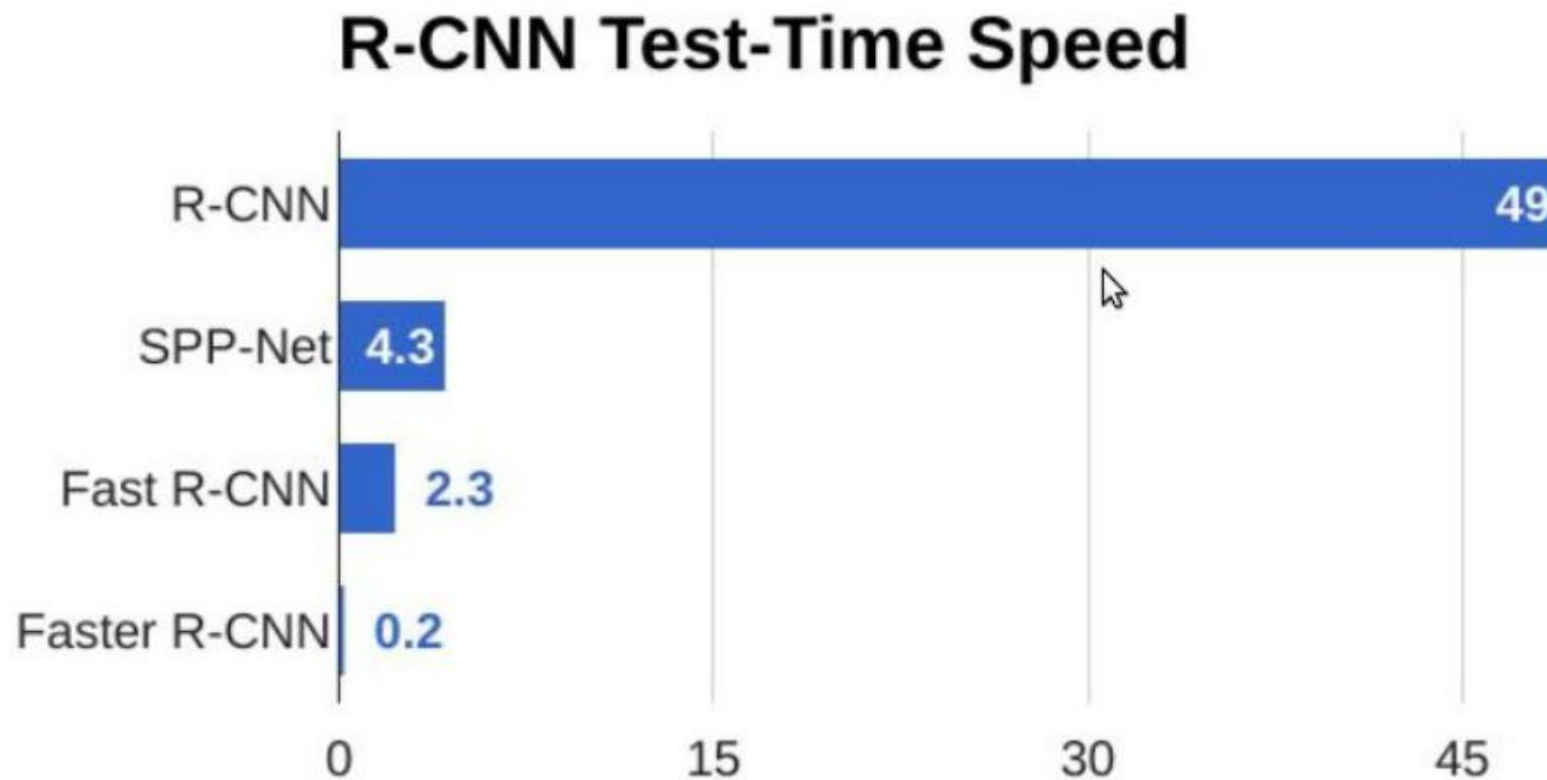
RPN testing:

- 1) Predict regions
- 2) Apply greedy non-maximum suppression
- 3) Select top 2000/300 by object presence probability

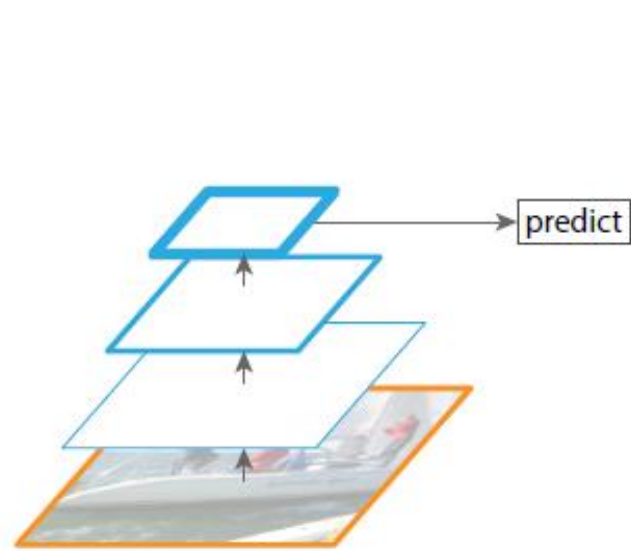
# Faster R-CNN (2015)



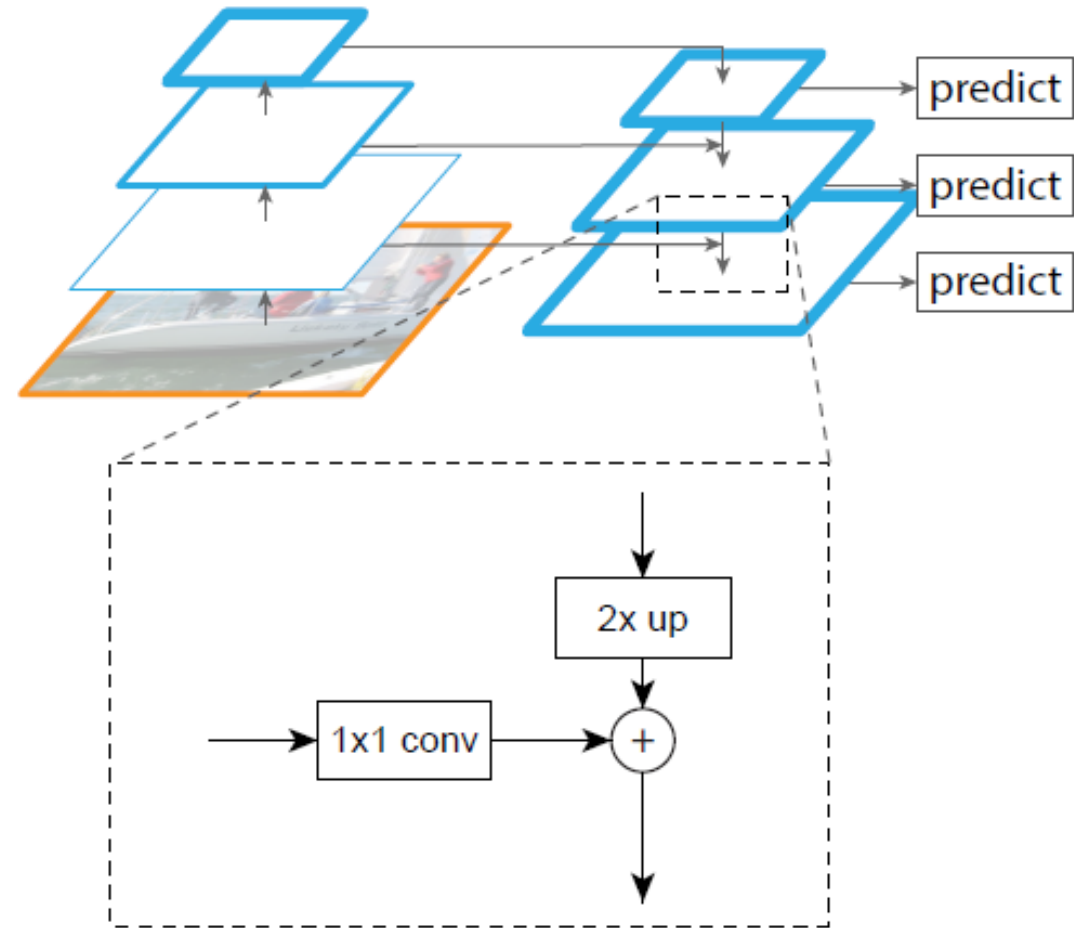
# Test time speed for R-CNN family



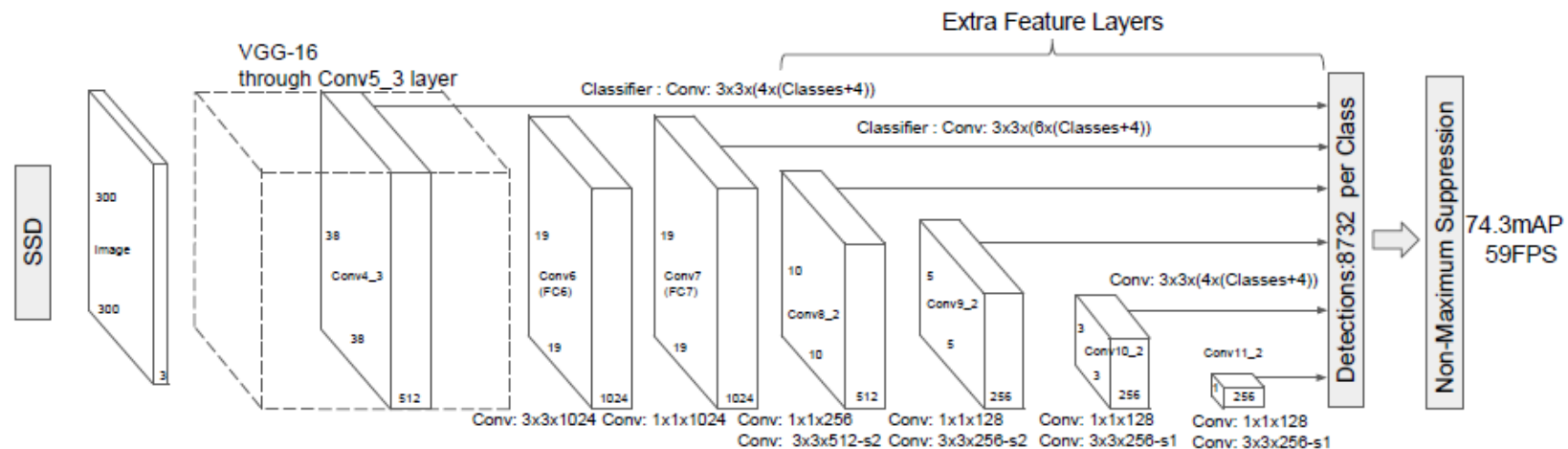
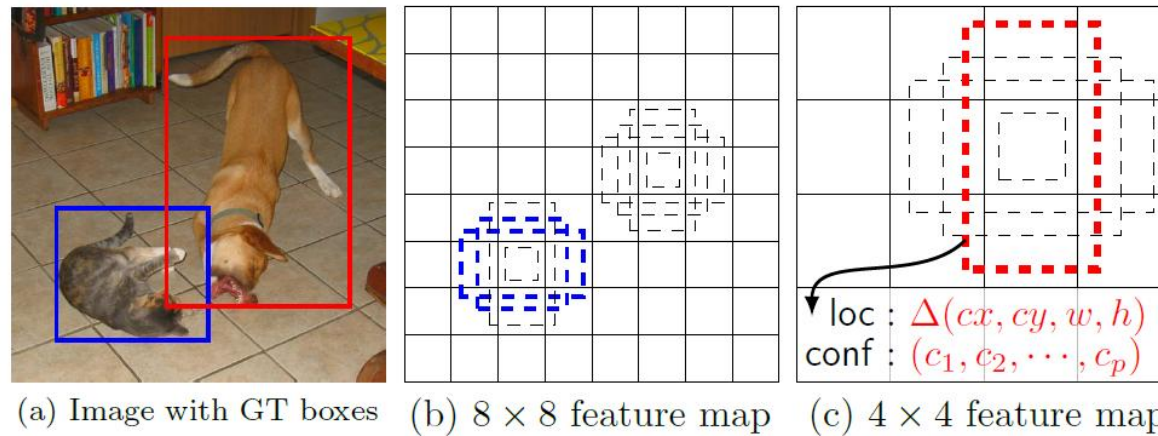
# Adding Feature Pyramid Network (FPN) to Faster R-CNN



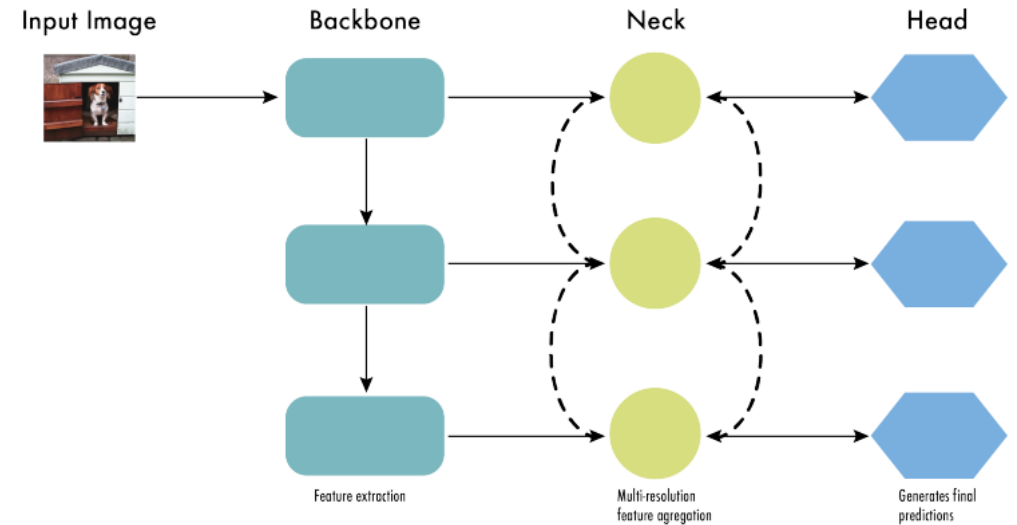
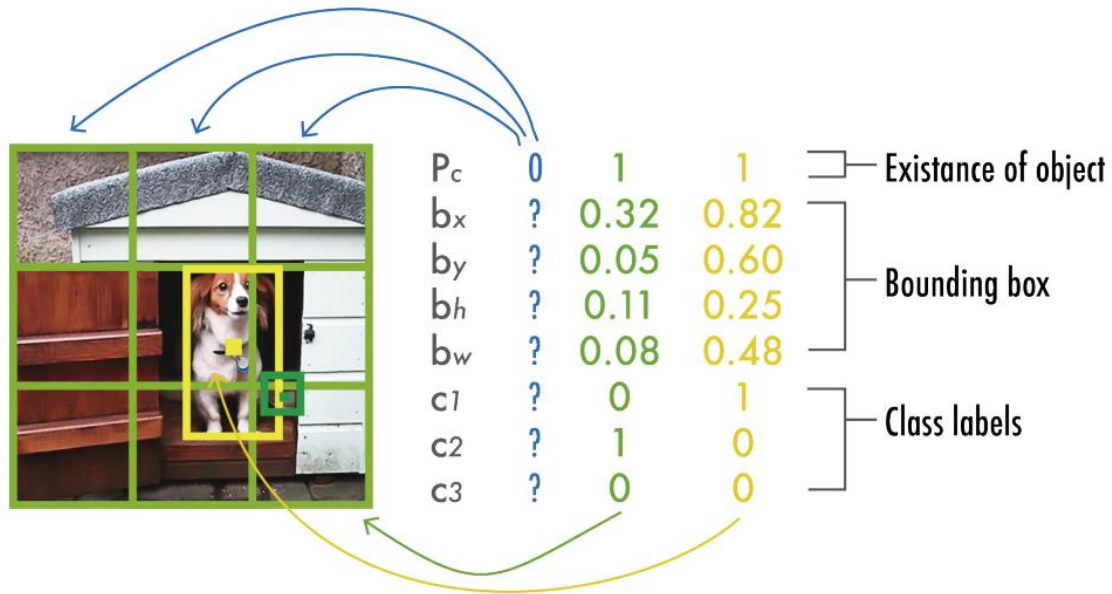
(b) Single feature map



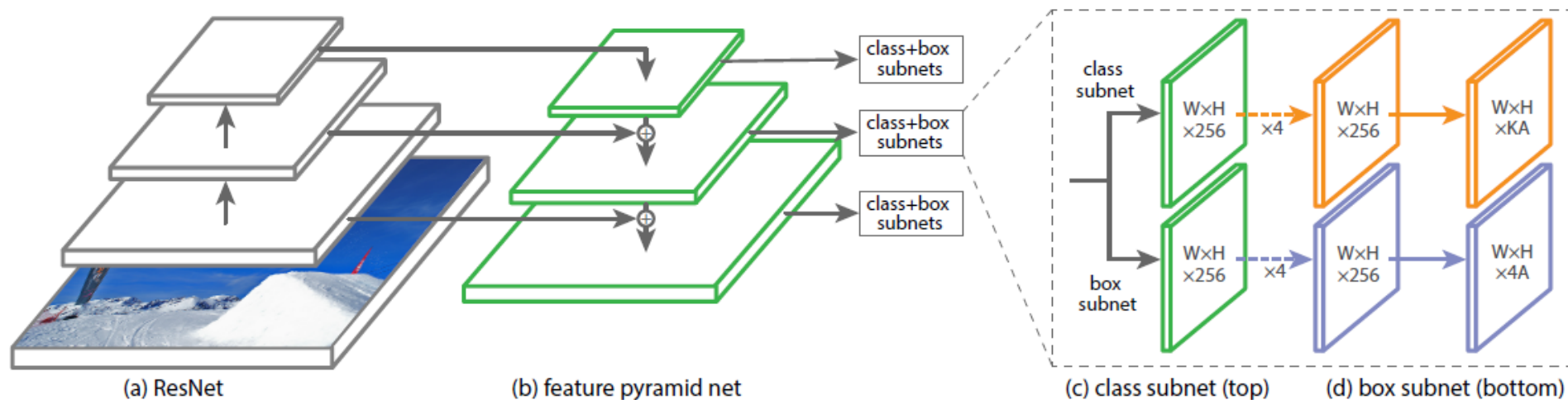
# One stage object detection



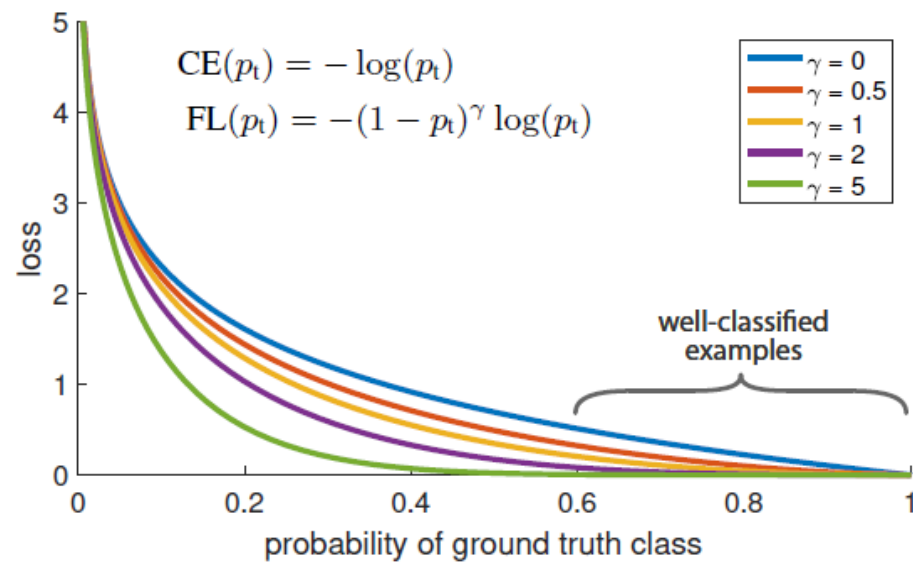
# YOLO family (v1, v2, ..., v12, ...)



# RetinaNet (2017)



## Focal loss



# Instance segmentation



# Mask R-CNN

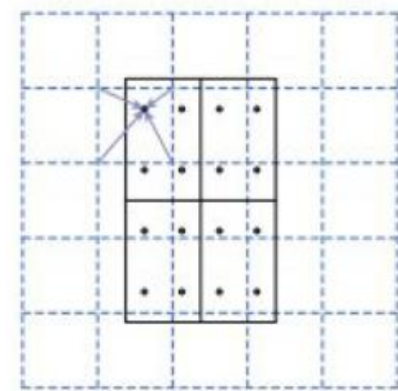
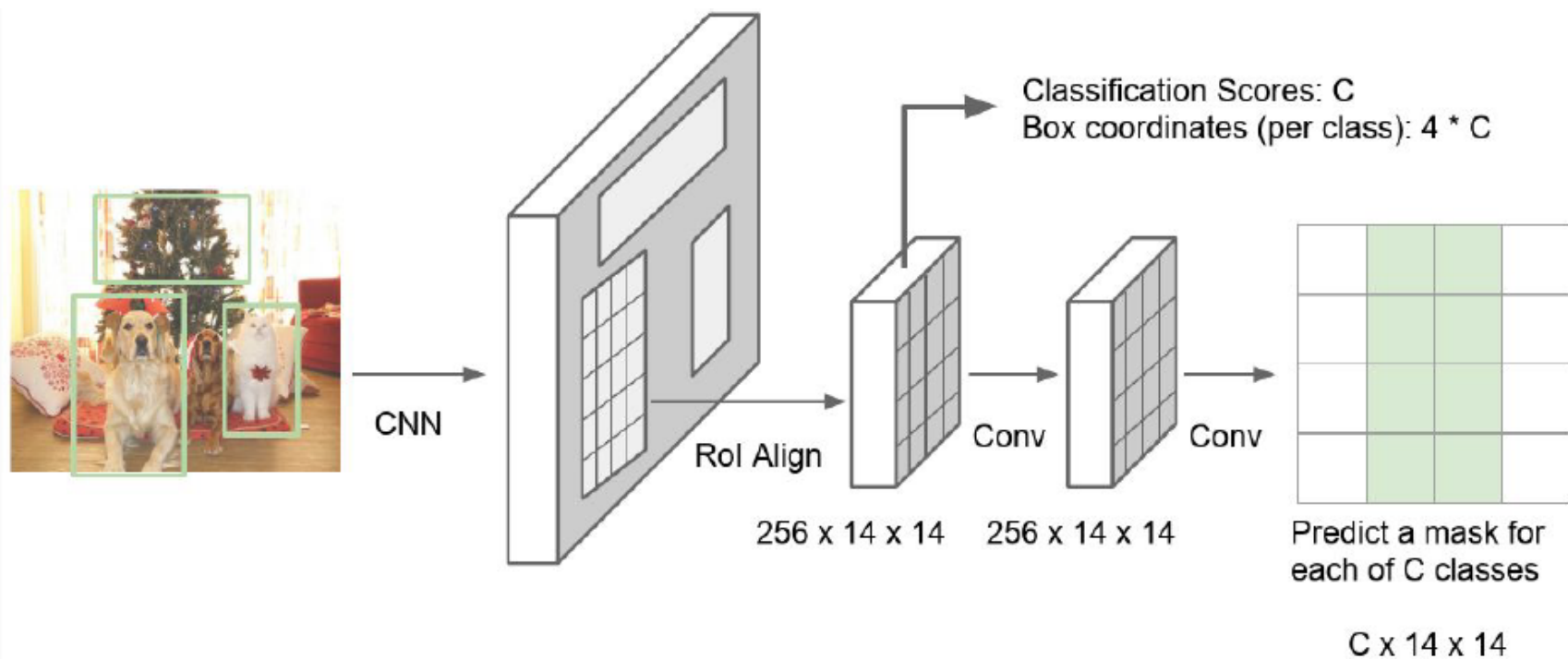


Image style transfer

# Image style transfer

- Inputs: content image, style image
- Output: stylized image



# Optimization-based ST (Gatys et al., 2015)

- Consider convolutional neural network (VGG)
- Content is reconstructed from activations on different layers
- Style is reconstructed from correlation of its activations on different layers

# Optimization-based ST

- Content loss (high level features should be similar):

$$\mathcal{L}_{content}^l(x, x_c) = \frac{1}{W_l H_l C_l} \left\| \Phi^l(x) - \Phi^l(x_c) \right\|_F^2$$

- Style loss for one layer (feature correlations should be similar):

$$\mathcal{L}_{style}^l = \frac{1}{C_l^2} \left\| G^l(x) - G^l(x_s) \right\|_F^2$$

- Total loss:

$$x = \arg \min_x \left\{ \mathcal{L}_{content}^{l_c}(x, x_c) + \alpha \sum_{l \in L_s} w_l \mathcal{L}_{style}^l(x, x_s) + \beta \mathcal{L}_{tv}(x) \right\}$$

# Optimization-based ST

- $\Phi_{cij}^l(x) \in \mathbb{R}^{C_l \times H_l \times W_l}$  - intermediate representation of  $x$  on layer  $l$ .
- Define Gram matrices  $\in \mathbb{R}^{N_l \times N_l}$ :

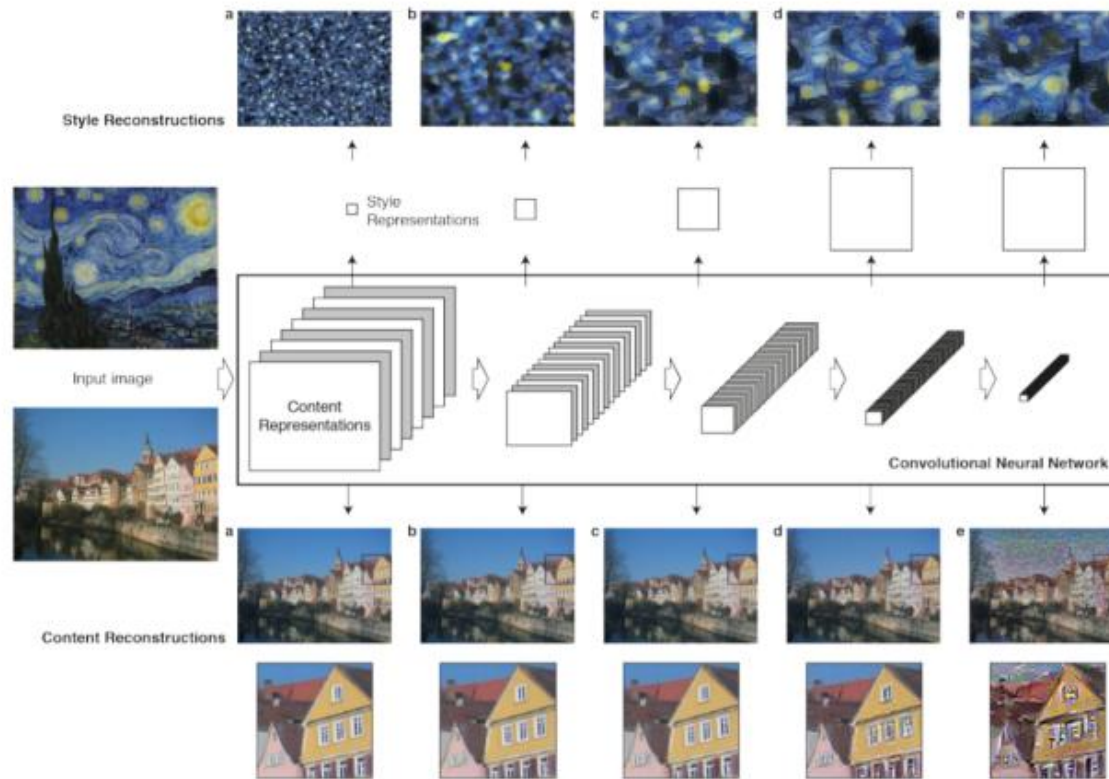
$$G_{uv}^l(x) = \frac{1}{H_l W_l} \langle \Phi_{u::}^l(x), \Phi_{v::}^l(x) \rangle = \frac{1}{H_l W_l} \sum_{i=1}^{H_l} \sum_{j=1}^{W_l} \Phi_{uij}^l(x) \Phi_{vij}^l(x)$$

- Total variation is a smoothness prior:

$$\mathcal{L}_{tv}(x) = \sum_{i,j} (x_{i+1,j} - x_{i,j})^2 + (x_{i,j+1} - x_{i,j})^2$$

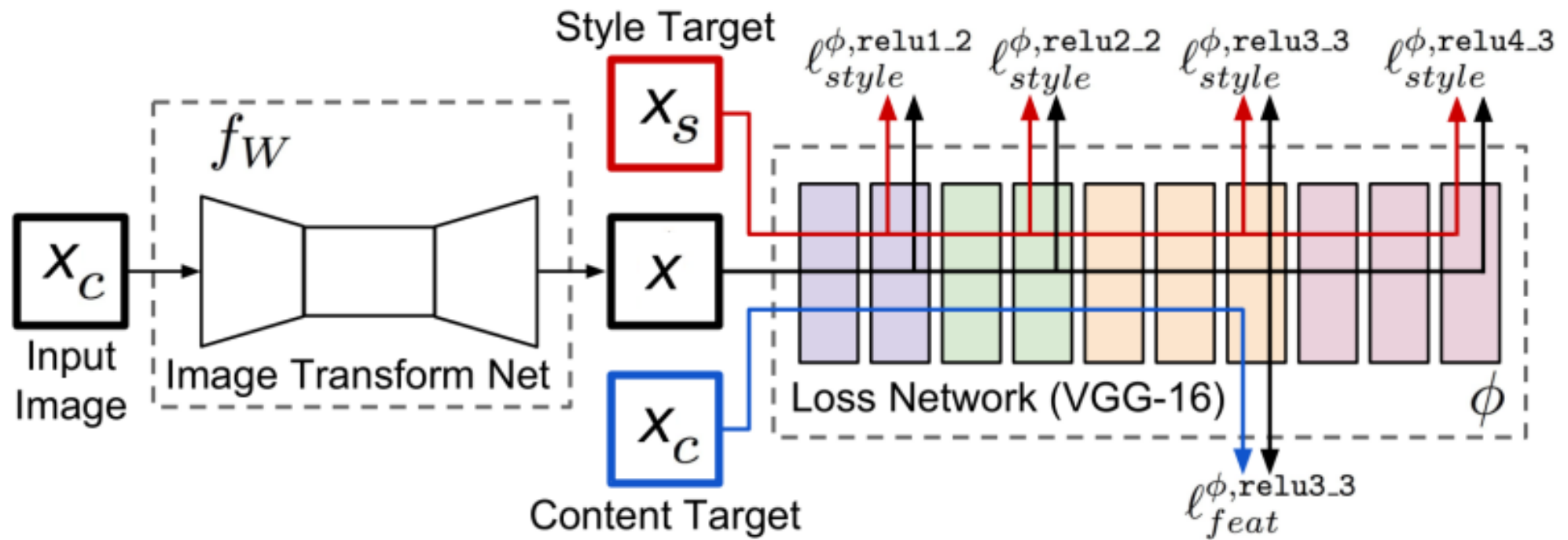
# Optimization-based ST

Content (lower row) and style reconstruction (upper row) from layers of increasing depth:



Deeper layers contain more abstract information.

# Transformation-based ST (Johnson et al., 2016)



# Transformation-based ST

| Image Size         | Gatys <i>et al</i> |         |         | Ours          | Speedup |      |              |
|--------------------|--------------------|---------|---------|---------------|---------|------|--------------|
|                    | 100                | 300     | 500     |               | 100     | 300  | 500          |
| $256 \times 256$   | 3.17               | 9.52s   | 15.86s  | <b>0.015s</b> | 212x    | 636x | <b>1060x</b> |
| $512 \times 512$   | 10.97              | 32.91s  | 54.85s  | <b>0.05s</b>  | 205x    | 615x | <b>1026x</b> |
| $1024 \times 1024$ | 42.89              | 128.66s | 214.44s | <b>0.21s</b>  | 208x    | 625x | <b>1042x</b> |